GUIDE 30 · EVENTS & EXPORTS

Run a scheduled export with cron, systemd or a Kubernetes CronJob

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=30-schedule-an-export. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** a saved query's full result written to a dated file on a schedule - a nightly CSV for finance, an hourly NDJSON feed for a data pipeline - with failures reported by the scheduler you already use.

QueryAPIGate doesn't schedule anything itself. `queryapigate export` runs one saved query and writes one file; cron, systemd or Kubernetes decide when, retry, and tell you when it failed.

Step 1: Run it once by hand

```shell
QUERYAPIGATE_HOME=/srv/queryapigate queryapigate export example_rentals_since \
  --param since=2026-10-01 --out '/exports/{name}_{date}.csv'
# Wrote 604 rows to /exports/example_rentals_since_2026-10-04.csv
```

- **No server, no API key.** It reads `queryapigate.db` in `QUERYAPIGATE_HOME` (or the PostgreSQL store in `QUERYAPIGATE_DATABASE_URL`) directly and connects to the database itself. Whoever can run it can already read the store, so no grant applies.
- **The published version** of the query runs - drafts never do.
- **`{name}` and `{date}`** (`YYYY-MM-DD`) in `--out` are filled in; missing directories are created.
- **`--format`** is `csv` (default), `tsv`, `ndjson` or `parquet`. The rows are streamed to the file, so a large result doesn't need a large amount of memory - see Export a large result.
- **`--param name=value`** (repeatable) supplies parameters, validated by the query's rules as over REST; `--connection` overrides the query's connection; `--timeout` overrides `QUERYAPIGATE_QUERY_TIMEOUT`.

The exact command for any query, with its parameters filled in, is on its **CLI** tab - see Get the exact CLI command for a saved query.

Step 2: Know how it fails

Every failure exits with status **1** and a one-line reason on stderr - verified:

```shell
queryapigate export: Invalid parameters: since must match the pattern \d{4}-\d{2}-\d{2}
queryapigate export: Saved query 'no_such' not found
queryapigate export: The query exceeded the time limit of 0.001 seconds and was cancelled
queryapigate export: Connection 'nope' not found
```

A failed run never leaves a broken file. The result is written to a temporary file and renamed into place only when it's complete. Verified: after the timeout and the bad connection above, the previous file at the same path was byte-for-byte unchanged.

Runs are recorded in run history like any other, with `key_name` `cli` - so a failing nightly export also shows up under the query's **History** tab, and counts toward its error-rate alert (which needs 10 recent runs to judge).

Two more things to know:

- **Same path, same day: replaced.** With `{name}_{date}`, a second run on the same day overwrites the first. Add the time to the name yourself if you run it more often (`--out "/exports/{name}_$(date +%H%M).csv"` - in a crontab, write each `%` as `\%`).
- **The scheduler's environment must have what the server's has**: `QUERYAPIGATE_HOME` (or `QUERYAPIGATE_DATABASE_URL`), `QUERYAPIGATE_SECRET_KEY` if passwords are encrypted, any `${VAR}` a connection's password refers to, and the database drivers. cron in particular starts with an almost empty environment.

Option A: cron

```cron
# m h dom mon dow   - every day at 06:00
QUERYAPIGATE_HOME=/srv/queryapigate
MAILTO=data-team@example.com
0 6 * * * /opt/queryapigate/venv/bin/queryapigate export example_rentals_since --out
'/exports/{name}_{date}.csv'
```

cron mails whatever a job writes when it fails (and when it succeeds - the "Wrote N rows" line). Redirect stdout to `/dev/null` to hear only about failures: `... > /dev/null`.

Option B: a systemd timer

`/etc/systemd/system/rentals-export.service`:

```ini
[Unit]
Description=Nightly rentals export

[Service]
Type=oneshot
User=queryapigate
EnvironmentFile=/etc/queryapigate/env
ExecStart=/opt/queryapigate/venv/bin/queryapigate export example_rentals_since --out
/exports/{name}_{date}.csv
```

`/etc/systemd/system/rentals-export.timer`:

ini

```
[Unit]
Description=Run the rentals export every morning

[Timer]
OnCalendar=*-*-* 06:00
Persistent=true

[Install]
WantedBy=timers.target
```

shell

```
sudo systemctl enable --now rentals-export.timer
systemctl list-timers rentals-export.timer
journalctl -u rentals-export.service        # each run's output, and why it failed
```

`Persistent=true` runs a missed export once the machine is back up. `EnvironmentFile` keeps the variables (and secrets) in one root-readable file. Verified the same command as a transient systemd timer (`systemd-run --user --on-active=2s ...`): it ran and wrote its file, with the output in the journal.

Option C: a Kubernetes CronJob

yaml

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: rentals-export
spec:
  schedule: "0 6 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: export
              image: ghcr.io/anantharajuc/queryapigate:0.48.0
              command: ["queryapigate", "export", "example_rentals_since", "--out",
"/exports/{name}_{date}.csv"]
              envFrom:
                - secretRef:
                    name: queryapigate-env      # QUERYAPIGATE_DATABASE_URL, QUERYAPIGATE_SECRET_KEY,
...
              volumeMounts:
                - name: exports
                  mountPath: /exports
          volumes:
            - name: exports
              persistentVolumeClaim:
                claimName: exports
```

Use a PostgreSQL store (`QUERYAPIGATE_DATABASE_URL`) here, so the job reads the same store as your running server without sharing a SQLite file between pods. A failed export fails the Job, which Kubernetes retries (`backoffLimit`) and reports like any other. (This manifest wasn't run for this guide; the command inside it was.)

With Docker Compose instead, run it in the existing container from the host's crontab: `docker compose exec -T queryapigate queryapigate export ...`.

Next steps

- Deliver a daily file to a partner's bucket - to S3, GCS or R2 instead of a local file, and only the rows that are new each time.
- Get the exact CLI command for a saved query.
- Export a large result without running out of memory - the same streaming, over HTTP.
- Group queries into a collection - `queryapigate collection export` moves query definitions between servers, a different kind of export.